

Relatório Técnico de Progresso

Semanas 1, 2, 3 e 4 — Engenharia de Dados, APIs & Visualização

Domínio: Música & Entretenimento

Autores: Francisco Sousa (Nº 53642) & João Nuno (Nº 51557)

Abordagem: Spec-Driven Development

Estado: Concluído (100%)

1. Contexto e Objetivos Gerais

O presente projeto consiste no desenho e implementação de um pipeline de dados modular do tipo **Extract, Transform, Load (ETL)** adaptado para a ingestão e processamento eficiente de grandes volumes de dados musicais em ambiente local. O ecossistema integra dados do *Spotify Million Playlist Dataset (MPD)*, da *Spotify Web API* e da *MusicBrainz API*, visando cruzar dados estruturais de consumo com metadados biográficos ricos para extrair insights de negócio sobre tendências sonoras globais.

2. Semana 1: Módulo de Extração (Extract)

Nesta fase infraestrutural, o foco central esteve no desenvolvimento de conectores robustos e tolerantes a falhas, capturando os dados em formato bruto para constituir a **Camada Bronze**.

2.1. Conectividade a APIs e Gestão de Restrições

- **Spotify Web API:** Implementação do fluxo de autenticação e recolha de perfis e preferências de artistas.
- **MusicBrainz API** (`extract_musicbrainz.py`): Desenvolvido com controlo rigoroso de *Rate Limiting* (atraso intencional de 1.1 segundos por chamada) e reaproveitamento de conexões HTTP via `requests.Session()`, mitigando erros prematuros de fim de ficheiro (TLS/SSL Handshake) e bloqueios por parte dos servidores.

2.2. Ingestão de Alto Volume por Chunks (Spotify MPD)

Para processar o milhão de playlists contido no dataset estático do MPD sem esgotar a memória volátil (RAM), o script `extract_mpd.py` foi desenhado para efetuar o *streaming* direto do arquivo `.zip`. Os dados são lidos em fatias ordenadas (chunks de 1.000 playlists), passando por um filtro imediato que descarta dados não relacionados com os artistas monitorizados, otimizando o armazenamento físico logo no ponto de entrada.

3. Semana 2: Módulo de Transformação (Transform)

A segunda semana marcou a conversão e higienização dos dados extraídos usando a biblioteca **Pandas**, estabelecendo a transição entre dados brutos e conjuntos analíticos refinados.

3.1. Arquitetura de Dados Medalhão (Medallion Architecture)

O fluxo de dados foi segmentado de forma lógica e física para garantir a reprodutibilidade e modularidade do pipeline:

CAMADA BRONZE Dados brutos em formato JSON isolados por fonte (originais de APIs e frações extraídas do ZIP).	CAMADA SILVER Tabela unificada e limpa resultante do achatamento (flattening) e do Left Join explícito usando a chave <code>artist_name</code> .	CAMADA GOLD (CSVS) Tabelas dimensionais agregadas, altamente leves e otimizadas para consumo analítico e visualização.
---	---	--

3.2. Controlo Automatizado de Data Quality

O script `transform_quality.py` atua como um portão de validação técnico, inspecionando o volume consolidado de **3.324.938 de linhas**. O resultado da auditoria foi registado nos seguintes termos:

Métrica de Qualidade (Transform)	Estado	Registo de Auditoria / Justificação Técnica
Linhas Duplicadas	PASS	0 registos redundantes encontrados. Confirmação de integridade no processo de achatamento.
Outliers de Tempo	PASS	0 faixas com duração superior a 30 minutos. Ruídos e ficheiros corrompidos eliminados.
Valores Nulos ('missing_values')	WARNING	Detetados nulos em dados biográficos (ex: <code>mb_end_date</code>). Validação humana conclui que a ausência de data final atesta que a esmagadora maioria dos artistas monitorizados continua no ativo, legitimando o dado.
Rácio de Matching	PASS	Apenas 3 artistas únicos (~54 linhas) falharam o emparelhamento de chaves com a MusicBrainz devido a divergências estritas de grafia (<0.001% de erro).

3.3. Sumarização Analítica (Camada Gold)

Para garantir o máximo desempenho na fase de visualização gráfica, a volumetria de mais de 3.3 milhões de linhas foi agregada em três visões estratégicas de escassos kilobytes:

- **Popularidade de Artistas ('dim_artist_popularity_gold')**: Identifica a dominância absoluta de artistas como *Drake* (847.160 aparições em 203.355 playlists) e o alcance expandido de seguidores.
- **Frequência de Géneros ('dim_genre_ranking_gold')**: Extração e explosão de listas de tags da MusicBrainz, mapeando um pódio liderado de forma destacada por *Hip Hop*, *Pop* e *Trap*.
- **Distribuição Geográfica ('dim_country_distribution_gold')**: Consolidação de faixas por país de origem do artista, demonstrando a concentração de consumo centrada no eixo americano (US) e canadiano (CA).

4. Semana 3: Módulo de Carregamento (Load)

A terceira semana marcou a transição final dos ficheiros planos soltos para uma infraestrutura de **Base de Dados Relacional e Analítica (OLAP)** baseada no motor **DuckDB**, consolidando o Star Schema estruturado para a fase de visualização.

4.1. Seleção Tecnológica e Engenharia de Performance

O motor **DuckDB** foi selecionado devido à sua arquitetura colunar nativa focada em queries analíticas e à sua capacidade de processar volumes massivos localmente com consumo otimizado de memória. A integração é realizada de forma direta no ficheiro de persistência física `data/gold/music_analytics.db`, eliminando a dependência de processos voláteis em RAM.

Durante o processo de carga, foi implementado um mecanismo de **Tipagem Robusta por Exceção**: as colunas de data biográfica (`mb_begin_date` e `mb_end_date`) foram mapeadas explicitamente como `VARCHAR`. Esta decisão de engenharia impediu falhas críticas de conversão geradas por anos truncados fornecidos pelas APIs (como o ano isolado 1969 do artista *Papillon*), conferindo resiliência ao pipeline.

4.2. Execução da Carga em Massa (Bulk Load)

O script `load_to_db.py` orquestrou o carregamento de forma síncrona através do leitor otimizado do DuckDB (`read_csv_auto`). Foram estruturadas e preenchidas as seguintes tabelas relacionais:

- `fact_playlists_tracks`: Tabela mãe de factos que absorveu a totalidade do schema estendido da camada Silver.
- `gold_artist_popularity`: Dimensão relacional contendo os índices consolidados de visibilidade dos artistas.
- `gold_genre_ranking`: Dimensão normalizada de tendências e categorias sonoras.
- `gold_country_distribution`: Tabela analítica de distribuição por soberania geográfica.

4.3. Auditoria de Data Quality Pós-Carga (SQL)

A validação de integridade foi executada em tempo real sobre o motor de base de dados através de asserções SQL explícitas, alcançando os seguintes resultados de conformidade técnica:

Métrica de Auditoria (Load)	Estado	Resultado Obtido / Evidência Técnica
Integridade Quantitativa de Carga	PASS	3.324.938 de 3.324.938 linhas inseridas e validadas sem truncagem ou perda de linhas (Duração: 7.25s).
Conformidade da Dimensão Artistas	PASS	26 registos analíticos de topo consolidados e indexados com chaves primárias estáveis.
Integridade Referencial (Chaves Estrangeiras)	PASS	0 registos órfãos detetados via query SQL de exclusão (LEFT JOIN). Coesão de chaves a 100%.

5. Semana 4: Módulo de Visualização e Storytelling (Visualization)

A quarta e última semana concretizou a entrega de valor do pipeline através do desacoplamento do sistema em microsserviços. Implementou-se um backend de serviço HTTP e um frontend visual interativo focado na extração e exposição dos padrões de consumo mapeados.

5.1. Arquitetura de Microserviços e Backend REST (FastAPI & Swagger)

Para mitigar o acoplamento rígido entre o cliente visual e o armazenamento físico, desenvolveu-se uma **API REST** autónoma recorrendo à framework FastAPI. O ficheiro `api.py` atua como uma camada intermédia de governança de dados: abre sessões de leitura controladas sobre o ficheiro do DuckDB e expõe os resultados estruturados em formato JSON.

A documentação e validação dos contratos da API foram automatizadas através da integração nativa da especificação OpenAPI, disponibilizando o ecossistema **Swagger UI** no endpoint `/docs`. As rotas geográficas foram otimizadas a nível de backend utilizando lógica SQL (`CASE WHEN`) para traduzir dinamicamente os códigos ISO de duas letras da MusicBrainz (ex: US, CA, PT) para os respetivos nomes por extenso legíveis (ex: Estados Unidos, Canadá, Portugal), refinando o consumo.

5.2. Interface Interativa e Componentes de Design (Streamlit & Plotly)

O frontend foi desenvolvido em `app.py` utilizando o ecossistema Streamlit. A interface consome os dados exclusivamente por via de pedidos HTTP síncronos (`requests`) à API, operando sem qualquer dependência ou conhecimento do ficheiro binário local do DuckDB. O design visual foi reestruturado de modo a suportar princípios avançados de legibilidade e *Data Storytelling*:

- **Containers e Métricas:** Agrupamento de KPIs macro com informação contextual sobre a volumetria processada da camada Silver.
- **Painéis Assimétricos Reativos:** Renderização paralela de gráficos colunares horizontais de géneros em eixos Plotly e gráficos de distribuição geográfica em formato circular (Donut Chart), permitindo a absorção imediata das conclusões.
- **Filtros Dinâmicos Laterais:** Componentes reativos na Sidebar que reconfiguram as consultas à API por Artista e País, recalculando instantaneamente as tabelas de detalhes e os diagramas.
- **Isolamento de Inspeção (Tabs):** A amostragem granular das linhas da tabela de factos foi isolada num separador expansível secundário, mantendo o foco do analista voltado para as respostas às perguntas de negócio.

5.3. Validação das Conclusões Analíticas face ao Enunciado

As visualizações interativas geradas pelo pipeline responderam empiricamente e de forma síncrona aos problemas formulados:

Pergunta Analítica (Enunciado)	Resposta Prática do Pipeline / Evidência Gráfica
Padrões de Género e Aceitação	Hegemonia Urbana: O painel de géneros quantifica o domínio absoluto do Hip Hop (2.08M), Pop (2.06M) e Trap (1.78M) na curadoria das playlists, revelando a forte preferência geracional por ritmos contemporâneos.
Diferenças entre Mercados Geográficos	Afunilamento Anglo-Saxónico: O gráfico regional comprova visualmente a centralização extrema da indústria analisada, em que o eixo dos Estados Unidos e Canadá retém mais de 80% de todas as faixas escutadas.
Atributos de Popularidade e Recorrência	Efeito Winner-Take-All: O gráfico de dispersão (Scatter Plot) demonstra o isolamento de mega-estrelas (com destaque para o artista Drake, com mais de 847 mil aparições), provando que uma minoria restrita retém o monopólio da audiência digital.

5.4. Higieneização Técnica do Histórico do Repositório

O portfólio do projeto foi encerrado com a reconfiguração total e aplicação do ficheiro `.gitignore`. Foram estabelecidas regras explícitas para o bloqueio de ficheiros físicos colunares de dados (`*.db`), ficheiros de variáveis de ambiente locais confidenciais (`.env`) e os diretórios automáticos de compilação do Python (`__pycache__`). A higienização foi garantida via terminal, desacoplando esses componentes do controlo de versões sem corromper ou eliminar os dados guardados localmente no disco físico.